

Introduction to Computer Programming

Prof. Alessio Martino

amartino@luiss.it

Department of Business and Management
Viale Romania 32, 00197 Rome



A.Y. 2024-2025

Outline

- 1 Collections
 - A Story of Two Collections
- 2 Dictionaries
 - Creating a Dictionary
 - Indexing and Manipulating a Dictionary
- 3 Dict Methods
 - `dict.keys()`, `dict.items()` and `dict.values()`
 - `dict.get()`
 - `dict.clear()`
 - `dict.pop()`
 - `dict.update()`
- 4 Iterating over Dicts
- 5 Tuples
- 6 Sets
- 7 Recap on Collections
- 8 References
- 9 Acknowledgments

List:

- a **linear ordered** collection of values
- list index their entities based on the **position** in the list.



Figure 1: A List

Dictionary:

- a **"bag of values"**, each with its own label
- the order of items in a dictionary is unpredictable
- dictionaries are indexed by keys (lookup tags)



Figure 2: A Dictionary

Dictionaries

- In Python, a dictionary is a mapping between a set of indices (*keys*) and a set of *values*
 - Items in a dictionary are **key-value pairs**
- Dictionaries are amongst the most powerful data collectors
- Dictionaries are also known as *hash tables* or *associative arrays*
 - They use a hash function for super-fast indexing of the keys
- Keys can't be any Python data type
 - Because keys are used for **indexing**, they should be **immutable**
 - Mutable objects do not support hashing
 - Common data types include strings and numbers
- Values can be any Python data type
 - Values can be **mutable** or **immutable**

There are two ways for creating a dictionary:

1 Step-by-step

```
>>> d1 = dict() # or d1 = {}, i.e., empty dictionary
>>> d1['money'] = 12 # add item
>>> d1['candy'] = 3 # add item
>>> d1['tissues'] = 75 # add item
>>> print(d1)
{'money': 12, 'candy': 3, 'tissues': 75}
```

- ② All-at-once

```
>>> d2 = {'money': 12, 'candy': 3, 'tissues': 75}
>>> print(d2)
{'money': 12, 'candy': 3, 'tissues': 75}
```

- Dictionary Query

```
>>> d = {'money': 12, 'candy': 3, 'tissues': 75}
>>> print(d['money'])
12
```

- Update the Dictionary

```
>>> d['candy'] = d['candy'] + 1
>>> print(d)
{'money': 12, 'candy': 4, 'tissues': 75}
```

- Add New Items to the Dictionary

```
>>> d['pen'] = 2
>>> print(d)
{'money': 12, 'candy': 4, 'tissues': 75, 'pen': 2}
```

- Delete Items from the Dictionary

```
>>> del(d['money'])                                   # del works on lists too...
>>> print(d)
{'candy': 4, 'tissues': 75, 'pen': 2}
```

Warning

If the index is not a key in the dictionary, Python raises a dedicated **KeyError** error.

```
>>> d = {'money': 12, 'candy': 3, 'tissues': 75}
>>> print(d['hello'])
```

KeyError

Traceback (most recent call last)

...[truncated output]

...[truncated output]

KeyError: 'hello'

Warning

The `in` operator behaves differently with respect to lists and strings:

- For offset-indexed sequences (e.g., strings and lists), `x in y` checks whether `x` is an item in the sequence `y`
- For dictionaries, `x in y` checks whether `x` is a key of the dictionary `y`

```
>>> L = [4, 6, 'hello', 7]           # a list
>>> 7 in L
True
>>> s = 'coding with python'        # a string
>>> 'p' in s
True
>>> d = {'money': 12, 'candy': 3, 'tissues': 75}  # a dict
>>> 'hello' in d
False
>>> 'money' in d
True
```

Dict Methods

dict.keys() returns a *view object*¹ containing all the keys of a dictionary. You can conveniently cast into a list, a more common and flexible data object.

```
>>> dishes = {'eggs': 2, 'sausage': 1,
              'bacon': 1, 'spam': 500}
>>> k = dishes.keys()                                # a view object (list-like)
>>> print(k)
dict_keys(['eggs', 'sausage', 'bacon', 'spam'])
>>> k = list(k)                                       # a proper list
>>> print(k)
['eggs', 'sausage', 'bacon', 'spam']
```

Warning

Dictionaries keep their items in the same order they were introduced²: **keys**, **values** and **items** (more on the latter later) are ordered according to this rule.

¹View objects are dynamic overview on the dictionary's entries

²In Python 3.6 and earlier, dictionaries are fully unordered.

`dict.values()` returns a *view object* containing all the values of a dictionary.
`dict.items()` returns a *view object* containing all the key-value pairs of a dictionary.

You can conveniently cast into a list, a more common and flexible data object.

```
>>> dishes = {'eggs': 2, 'sausage': 1,
               'bacon': 1, 'spam': 500}
>>> v = dishes.values()           # a view object (list-like)
>>> v = list(v)                   # a proper list
>>> print(v)
[2, 1, 1, 500]
>>> i = dishes.items()            # a view object (list-like)
>>> i = list(i)                   # a list of key-value tuples
>>> print(i)
[('eggs', 2), ('sausage', 1),
 ('bacon', 1), ('spam', 500)]
```

```
>>> dishes = {'eggs': 2, 'sausage': 1,
               'bacon': 1, 'spam': 500}
>>> v = dishes.get('eggs', 10)           # 'eggs' exists in dishes
>>> print(v)
2
>>> v = dishes.get('water', 10)          # 'water' does not exist in dishes
>>> print(v)
10
```

Since `dict.get()` relies on a default value if `key` is not found, it will never raise a `KeyError`.

Hint

Pattern of usage: check whether a key is already in a dictionary and assuming a default value if the key is not there.

```
>>> dishes = {'eggs': 2, 'sausage': 1,
              'bacon': 1, 'spam': 500}
# 'eggs' exists in dishes, so we take its value and add + 1
>>> dishes['eggs'] = dishes.get('eggs', 0) + 1
>>> print(dishes)
dishes = {'eggs': 3, 'sausage': 1,
          'bacon': 1, 'spam': 500}
# 'water' does not exists in dishes
# so we plug the default value 0 and add + 1
>>> dishes['water'] = dishes.get('water', 0) + 1
>>> print(dishes)
dishes = {'eggs': 3, 'sausage': 1,
          'bacon': 1, 'spam': 500, 'water': 1}
```

dict.get() explained:

Let d be a dictionary. Saying

$v = d.get(k, x)$

is a shortcut for:

if k in d :

$v = d[k]$

else:

$v = x$

Similarly, saying

$d[k] = d.get(k, x) + y$

is a shortcut for:

if k in d :

$d[k] = d[k] + y$

else:

$d[k] = x + y$

`dict.clear()` deletes all items from a dictionary.

```
>>> dishes = {'eggs': 2, 'sausage': 1,
               'bacon': 1, 'spam': 500}
>>> dishes.clear()
>>> print(dishes)
{}                                     # an empty dictionary
```

Note

There is a strong analogy with lists:

- `clear()` also works on lists (same effect → empty list).

`dict.pop(key)` deletes the dictionary entry corresponding to the key `key` and returns its value.

```
>>> dishes = {'eggs': 2, 'sausage': 1,
              'bacon': 1, 'spam': 500}
>>> v = dishes.pop('sausage')
>>> print(dishes)
{'eggs': 2, 'bacon': 1, 'spam': 500}
>>> print(v)
1                                     # former value for 'sausage'
```

Note

There is a strong analogy with lists:

- `pop()` also returns the value when called on a list;

```
>>> d1={1: 10, 2: 20}
>>> d2={3: 30, 4: 40}
>>> d1.update(d2)
>>> print(d1)
{1: 10, 2: 20, 3: 30, 4: 40}    # d1 is modified in-place, d2 is unchanged
```

If a key exists both `dict` and `other_dict`, the key-value pair in `other_dict` overwrites the one in `dict`.

```
>>> d1={1: 10, 2: 20}
>>> d2={3: 30, 2: 40}    # we have an overlapping key (i.e., 2)
>>> d1.update(d2)
>>> print(d1)
{1: 10, 2: 40, 3: 30}    # the value associated to key 2 is taken from d2
```

Iterating over Dicts

You can iterate over a dictionary with a very simple `for` loop:

- over its keys (way common)
- over its values (less common)

Hint

You can generate the sequence of keys and values with `dict.keys()` and `dict.values()`, respectively.

```
dishes = {'eggs': 2, 'sausage': 1, 'bacon': 1, 'spam': 500}
for k in dishes.keys():
    print("Key is:", k, "and Value is:", dishes[k])
```

Hint

If you iterate over keys, it's trivial to have both key and value. If you iterate over values, it's not trivial to get the corresponding key.

You can iterate **simultaneously** over keys and values of a dictionary with a very simple `for` loop.

Hint

You can get the key-value pairs with `dict.items()`.

```
dishes = {'eggs': 2, 'sausage': 1, 'bacon': 1, 'spam': 500}
for k, v in dishes.items():
    print("Key is:", k, "and Value is:", v)
```

What's going on here?

- there are **two iteration variables**, namely `k` and `v`
- `dict.items()` generates a list of key-value pairs (tuple)
- the first loop variable, namely `k`, contains the key in the key-value pair
- the second loop variable, namely `v`, contains the corresponding value in the key-value pair

Tuples

Tuples, like lists, are **ordered collections** of items.

You can create a tuple by enclosing items in round brackets:

```

>>> t1 = (90, 98, 95)           # homogeneous tuple
>>> t2 = (90, 'hello', 95, [1, 2]) # heterogeneous tuple
>>> t3 = ()                     # empty tuple
  
```

or, you can omit the parenthesis altogether³

```

>>> t1 = 90, 98, 95             # homogeneous tuple
>>> t2 = 90, 'hello', 95, [1, 2] # heterogeneous tuple
  
```

Warning

When you create a singleton tuple, you must include a trailing comma!

```

>>> t = (90)    # type(t) yields 'int'
>>> t = (90,)   # now we have a 1-element tuple
  
```

³This is called *tuple packing*.

Tuples (and lists) can also be *unpacked*⁴.

```
>>> t = (90, 0, 'hello')
>>> a, b, c = t           # unpacking syntax
```

which is basically the same as doing

```
>>> a = t[0]
>>> b = t[1]
>>> c = t[2]
```

and therefore we will have:

```
a → 90
b → 0
c → 'hello'
```

⁴Unpacking is when you assign values from a tuple/list to a sequence of variables.  20/30

Tuples support all of the common operations we have for lists, notably:

- indexing
`t=(98, 7, 'hello')` `t[1]` yields 7
- slicing
`t=(98, 7, 'hello', 'world')` `t[1:3]` yields (7, 'hello')
- the `+` operator for concatenation
`t1=(98, 7)` and `t2=('hello',)` `t1+t2` yields (98, 7, 'hello')
- the `*` operator for replication
`t=(98, 7)` `t*3` yields (98, 7, 98, 7, 98, 7)
- the `in` (and `not in`) operator for membership checking
`t=(98, 7)` `'hello' in t` yields `False`
- the `len()` function to calculate the length of a tuple

And you can also iterate over tuples, e.g. via `for` loops:

```
t = (98, 7, 'hello')
for item in t:
    print(item)
```

```
>>> t = (98, 7, 'hello')
>>> t[-1] = 'python'
```

So you cannot change a tuple, just like you cannot change a string.
Workarounds:

- As the latter is concerned:

```
>>> t = (98, 7, 'hello')    # t is a tuple
>>> L = list(t)             # from tuple to list
>>> print(L)
[98, 7, 'hello']
>>> L.append('python')      # do some manipulations
>>> t = tuple(L)            # from list to tuple
>>> print(t)
(98, 7, 'hello', 'python')
```

Sets

A set is an unordered (unindexed) collection of items with no repetitions.

You can create an empty set thanks to the `set()` function.

And you can create a non-empty set thanks to the `set()` function or curly brackets.

```
# empty set
>>> s1 = set()
# non-empty set with curly brackets
>>> s2 = {'a', 1}
# non-empty set with set()
>>> s3 = set(['a', 1])
```

Warning

To not confuse the curly brackets to create dicts with the curly brackets to create sets.

A set can contain only hashable items, notably:

- numbers
- strings
- tuples only if they do not contain mutable objects

and in no way they can contain mutable objects, notably:

- dictionaries
- lists
- tuples that contain mutable objects

In Python, the majority of immutable objects are also hashable.

Functions for sets:

- membership
 $x \text{ in } S$
- the `len(s)` function to calculate the cardinality (size) of a set `s`

Operations between sets:

- union (new set with elements in either `s1` or `s2`)
 $s1 \mid s2$ or `s1.union(s2)`
- intersection (new set with elements in both `s1` and `s2`)
 $s1 \& s2$ or `s1.intersection(s2)`
- difference (new set with elements in `s1` but not in `s2`)
 $s1 - s2$ or `s1.difference(s2)`
- symmetric difference (new set with elements in `s1` or `s2`, but not both)
 $s1 \wedge s2$ or `s1.symmetric_difference(s2)`

Note

Set operations yield a brand new set.

Set methods:

- `s.add(x)`
adds `x` to the set `s` (if `x` already in `s`, nothing happens)
- `s.remove(x)`
removes `x` from the set `s` (if `x` is not in `s`, a `KeyError` is raised)
- `s.discard(x)`
removes `x` from the set `s` (if `x` is not in `s`, nothing happens)
- `s.update(x)`
adds elements from `x` into the set `s` (`x` can be any iterable, e.g., list, tuple or another set)
- `s.clear()`
removes all elements from `s`
- `s.pop()`
removes (and returns) an arbitrary item from `s`

Note

Set methods change the set in-place.

Recap on Collections

- `list` and `tuple` keep order; `set` and `dict` do not
- `dict` associates a key with a value; `set`, `tuple` and `list` contain just values
- `set` and `dict` keys only require items to be hashable; `list`, `tuples` and `dict` values do not
- `set` and `dict` keys forbid duplicates; `list`, `tuples` and `dict` values allow duplicates.

Checking for membership inside a `set` or `dict` is super-fast thanks to hashing.
`tuple` and `list` require scanning the list/tuple until the element is found.

References

- Gutttag, J. V. (2013). *Introduction to Computation and Programming using Python (Revised and Expanded Edition)*. MIT Press. [Sections 5.1, 5.4 & 5.5]
- Downey, A. (2015). *Think Python*. Green Tea Press ed. 2nd. [Chapters 11 & 12]
- <https://docs.python.org/3/library/stdtypes.html#dict>
- <https://docs.python.org/3/library/stdtypes.html#typeseq-common>

Acknowledgments